



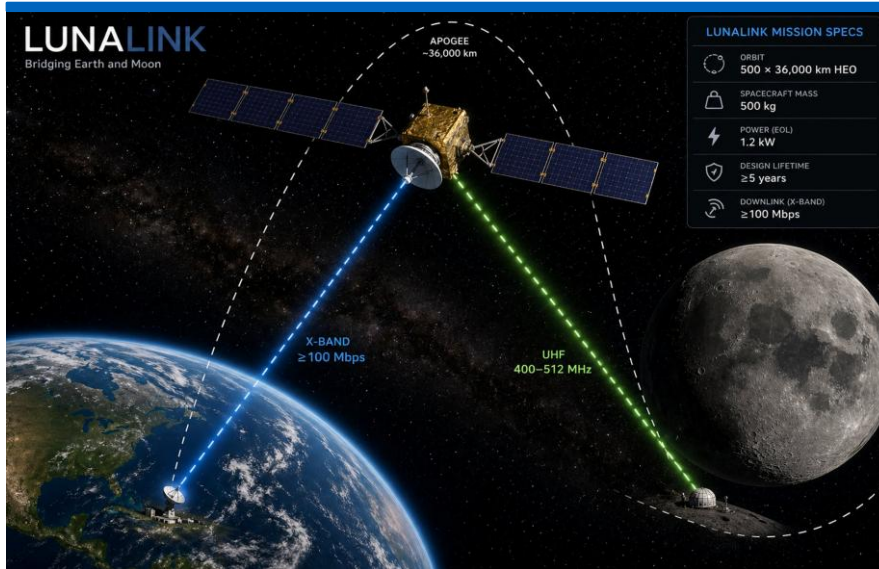
Project X

LunaLink Simulation Challenge

Spacecraft Design: Fundamentals — SS 2026

Individual project · Python · AI-assisted · Prize awarded

The mission: LunaLink



Parameter	Value
Orbit type	HEO — Molniya-type (500 × 36,000 km)
Inclination	63.4°
Orbital period	≈ 12 hours
S/C mass	500 kg
Power budget (EOL)	1.2 kW
Design lifetime	≥ 5 years
Earth downlink	X-band · ≥ 100 Mbps
Moon uplink	UHF · 400–512 MHz
Envelope (approx.)	2.0 × 1.5 × 1.0 m (box)
Ground station	Ottobrunn, Germany (48.07° N, 11.65° E)
Min. elevation	5°

All mission parameters above are fixed. Your subsystem design and simulation must be consistent with them.

What you build — and the rules



Python simulation

Sizing + orbit propagation + subsystem simulation over ≥ 3 orbits. Clean, commented, runnable with one command.



GUI with visualisation

Interactive interface. Parameters adjustable by the user. Live plots update accordingly.
Built in Python (Tkinter / PyQt / Streamlit / Gradio).



Report (max 10 pages)

Introduction · Assumptions · Results · How you used AI · References.
PDF format.

Individual project — each student works alone

Python only (any libraries). Code must run on a clean install with pip.

Simulation must cover at least 3 full orbits (≥ 36 hours of simulated time)

Make your own assumptions — state every one of them explicitly

Choose one subsystem to simulate

Each subsystem is equally valid. Pick the one that interests you most — all involve orbit propagation + sizing + time-series simulation + GUI.

EPS — Electrical Power



- Define power modes
- Propagate orbit + assume pointing
- Size solar arrays + batteries
- Simulate energy variation over 3+ orbits
- GUI: change duty cycle, see budget

Thermal Control



- Assume coating, pointing, materials, power modes
- Lumped-parameter thermal model
- Simulate orbit + temperature variation of surfaces
- Check internal device temperatures
- GUI: change coating, see response

ADCS



- Assume mechanical properties of satellite
- Size magnetorquers + reaction wheels
- Define attitude modes (detumbling, sun-pointing,...)
- Simulate attitude over 3+ orbits
- GUI: visualise orientation / angular momentum

TT&C



- Assume gains, losses, antenna position
- Define link budget (Earth + Moon; Uplink and Downlink)
- Simulate contact windows + data volume
- Comm windows: Moon relay $\leftrightarrow$ Earth GS
- GUI: show ground track + contact windows



EPS

— Electrical Power System

Step-by-step scope

- 1 Define power modes (safe, nominal, peak) and power budget per mode
- 2 Propagate HEO orbit and assume solar panel pointing strategy
- 3 Size solar array area (EOL) and battery capacity (DoD, Wh)
- 4 Simulate energy variation and power generation/consumption for ≥ 3 orbits
- 5 Build GUI: adjust payload duty cycle and power mode, see budget live

Expected outputs

- Time-series plot: power generation vs consumption (W) over 3 orbits
- State-of-charge plot (%) — must show eclipse discharge and recharge
- Solar array area (m²) and battery capacity (Wh) sizing results
- GUI slider: payload power on/off, see energy impact in real time
- Power budget table for different modes

Key assumption to state: Solar panel efficiency, pointing loss, degradation rate (%/yr), battery chemistry, DoD limit, power modes (safe / nominal / peak relay 1.2 kW)



TCS

— Thermal Control System

Step-by-step scope

- 1 Assume coating, pointing, materials and active power mode per orbit phase
- 2 Build lumped-parameter thermal model: satellite box + key internal nodes
- 3 Simulate orbit and compute external heat fluxes (solar, albedo, Earth IR)
- 4 Simulate surface temperature variation and selected internal device temperatures
- 5 Build GUI: change surface coating / internal dissipation, see temperature response

Expected outputs

Time-series: external surface temperature (K) over 3 orbits — all 6 faces

Internal node temperatures. You decide some critical ones

Hot case / cold case equilibrium temperatures assumptions

GUI dropdown: select coating material, see temperature envelope update

Flag: which orbit phases risk exceeding component temperature limits

Key assumption to state: Absorptivity / emissivity of chosen coating, satellite specific heat capacity, internal power dissipation per mode, eclipse fraction at perigee vs apogee



ADCS

— Attitude Determination and Control

Step-by-step scope

- 1 Assume mechanical properties: mass, moments of inertia, CoM/CoP offset
- 2 Size magnetorquers (dipole moment) and reaction wheels (Nms, Nm, rpm)
- 3 Define attitude modes: detumbling, safe mode, sun pointing, Earth pointing,...
- 4 Simulate detumbling manoeuvre and another ADCS mode (i.e.; sun-pointing mode) over ≥ 3 orbits
- 5 Build GUI: visualise satellite orientation and angular momentum over orbit

Expected outputs

- Time-series: angular velocity (deg/s) — detumbling convergence plot
- Reaction wheel angular momentum (Nms) — show desaturation triggers
- Disturbance torques magnitude (Nm) vs orbit position (gravity gradient, SRP, drag)
- Attitude angle error (deg) — sun-pointing accuracy over multiple orbits
- GUI 3D view: satellite orientation quaternion animated along the orbit

Key assumption to state: Initial tumble rate (e.g. 10 deg/s), principal moments of inertia (from box envelope + 500 kg), B-field model (IGRF or dipole), SRP coefficient



TT&C

— Telemetry, Tracking and Command

Step-by-step scope

- 1 Assume antenna gains, pointing losses, noise temperature, implementation losses
- 2 Define link budget: Earth downlink (X/X-band / UHF) and Moon relay (X-band / UHF)
- 3 Propagate orbit and compute slant range + elevation angle vs time
- 4 Simulate contact windows (Earth GS + Moon) and data volume per contact
- 5 Build GUI: show ground track + Earth/Moon contact windows on a map

Expected outputs

- Link budget table: EIRP, path loss, SNR, margin (≥ 3 dB)
— Earth and Moon
- Time-series: slant range (km) + elevation angle (deg) over 3 orbits
- Contact window timeline: start/end times, duration, data volume (Mbit/contact)
- Total data budget: how much science data can be downlinked to Earth
- GUI map: ground track + Ottobrunn visibility cone + Moon comm window overlay

Key assumption to state: Antenna diameter and efficiency, transmit power (W), noise temperatures (satellite + ground), pointing loss, atmospheric loss (rain fade at X-band / UHF)

Report and submission requirements

Report structure (max 10 pages, PDF)

1

Introduction

Mission context, chosen subsystem, scope of your simulation

2

Assumptions

Every engineering assumption stated explicitly and justified

3

Results

Key sizing outputs + simulation plots with physical discussion

4

How you used AI

Which tools, for which tasks — and what you had to correct or verify

5

References

All sources cited (SMAD, papers, datasheets, websites)

Code submission

Folder structure

One zip: /code/ (your scripts) + /report/ (PDF) + README.txt

README.txt

How to install dependencies (pip install -r requirements.txt)
and how to run the simulation and GUI

Simulation script

main_simulation.py — produces all plots without GUI. Must run headless.

GUI script

main_gui.py — launches the interactive interface

Submission deadline: 8 July 2026, 23:59 — via Moodle (one zip file, max 50 MB)

Winning criteria, prize and AI policy

How winners are selected

35%

Physical correctness

Sizing equations correct, outputs in right order of magnitude, margins respected

25%

Visualisation quality

Plots are clear, labelled, physically meaningful. GUI works and adds insight.

25%

Engineering judgment

Assumptions stated and justified. Trade-offs discussed. You understand your own results.

15%

User-friendliness

README is clear, code runs first time, GUI is intuitive without instructions



There's a prize

Does not affect your grade

Awarded at the final session · Top projects shared with next year's cohort



Use AI critically

- Use any AI tool freely: ChatGPT, Claude, Copilot, GitHub CoPilot
- AI will also be used to check your results for physical correctness
- You must describe how you used AI in the report (Section 4)
- The real skill: knowing when the AI is wrong — and fixing it